# Two-Branch Deeper Graph Convolutional Network for Hyperspectral Image Classification

Linzhou Yu, Jiangtao Peng, *Senior Member, IEEE*, Na Chen, Weiwei Sun, *Senior Member, IEEE*, and Qian Du, *Fellow, IEEE*

***Abstract*— The graph convolutional network (GCN) has recently attracted great attention in hyperspectral image (HSI) classification due to its strong ability to aggregate information of neighborhood nodes. However, a GCN model usually suffers from the oversmoothing problem (i.e., all nodes' representations converge to a stationary point) when the number of GCN layers is increased. In addition, GCNs always work on superpixel-level nodes to reduce the computational cost, so pixel-level features cannot be well captured. To deal with these problems, a novel two-branch deeper GCN (TBDGCN) is proposed to combine the advantages of superpixel-based GCN and pixel-based CNN, which can simultaneously extract superpixel- and pixel-level features of HSIs. In the GCN branch, a GCN module with the DropEdge technique and residual connection is designed to alleviate oversmoothing and overfitting problems, which results in a deeper network structure with more than ten layers. In the CNN branch, to capture spatial positional information and channel information, a mixed attention mechanism is constructed to extract attention-based spectral–spatial features. The features of the GCN and CNN branches are then fused for classification. Experimental results on three benchmark HSI datasets show that the classification performance of our TBDGCN is better than existing GCN models, especially in the case of a small sample size.**

***Index Terms*— Attention mechanism, convolutional neural network (CNN), DropEdge, graph convolutional network (GCN), hyperspectral image (HSI) classification.**

## I. Introduction

HYPERSPECTRAL images (HSIs) contain rich spectral and spatial information of observed objects [1], [2]. Compared with RGB and multispectral images, an HSI consists of hundreds of continuous spectral bands, which can be used to detect and recognize subtle differences between similar materials. Nowadays, HSIs have been successfully applied in many fields, such as precision agriculture [3], military field [4], and so on [5]. The objective of HSI classification is to assign a label to each pixel in the image. In the past few decades, many machine learning methods have been introduced for HSI classification, such as $k$-nearest neighbors, support vector machines (SVMs) [6], logistic regression [7], and random forest (RF) [8]. These methods only use spectral information, and their classification performance is limited. To increase the classification accuracy, the spectral and spatial information is jointly used, and many spatial–spectral classification methods are proposed, such as the composite kernel [9], [10], the Markov random field (MRF) method [11], Gabor filtering [12], [13], the morphological method [14], and joint sparse representation [15].

Although these traditional classification algorithms have produced good classification results, they are always based on handcrafted features. These handcrafted features may be suitable for some specific data but are not data-adaptive. Different from traditional methods, deep learning can automatically extract features with an end-to-end learning model by using neural networks, which has achieved success in many fields, especially for image classification [16]. Chen et al. [17], [18] first introduced the deep belief network (DBN) and convolutional neural networks (CNNs) for HSI classification. A CNN has the characteristics of local connection and weight sharing, which can not only effectively extract the local features of the image but also significantly reduce the number of parameters. Due to its superior performance, many CNN-based methods have been proposed for HSI classification, such as the dual-channel CNN model [19], the 3-D CNN model [20], HybridSN [21], and LWCNN [13]. In the CNN model, when the depth of the network is increased, it is likely to arise gradient disappearance or gradient explosion problems. To solve these problems, a spectral–spatial residual network (SSRN) [22] was proposed. Although CNNs have shown excellent performance for HSI classification, there are still some challenges. For example, a CNN is designed to handle regular Euclidean data, i.e., the coordinates of the underlying data representation have a grid structure [23], so it is not suitable for data with irregular geometrical distributions. In addition, CNN-based methods often have the overfitting problem in the case of small samples [24].

To deal with the aforementioned problems, graph convolutional networks (GCNs) have attracted great attention [25], [26]. GCNs have the ability to process data of irregular distribution effectively for they can aggregate the node features

Manuscript received 23 July 2022; revised 30 December 2022; accepted 12 March 2023. Date of publication 15 March 2023; date of current version 28 March 2023. This work was supported in part by the National Natural Science Foundation of China under Grant 42171351, Grant 42122009, and Grant 41971296; and in part by the Natural Science Foundation of Hubei Province under Grant 2021CFA087. *(Corresponding authors: Jiangtao Peng; Weiwei Sun.)*

Linzhou Yu, Jiangtao Peng, and Na Chen are with the Hubei Key Laboratory of Applied Mathematics, Faculty of Mathematics and Statistics, Hubei University, Wuhan 430062, China (e-mail: pengjt1982@hubu.edu.cn).

Weiwei Sun is with the Department of Geography and Spatial Information Techniques, Ningbo University, Ningbo 315211, China (e-mail: sunweiwei@nbu.edu.cn).

Qian Du is with the Department of Electrical and Computer Engineering, Mississippi State University, Mississippi State, MS 39762 USA (e-mail: du@ece.msstate.edu).

Digital Object Identifier 10.1109/TGRS.2023.3257369

1558-0644 © 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 01:46:42 UTC from IEEE Xplore. Restrictions apply.

in a graph and capture high-order characteristics of data structure [25]. In the field of HSI classification, Qin et al. [27] proposed a semisupervised spectral–spatial GCNs ($S^2$GCNs), which used the adjacent nodes in the graph to approximate the convolution. The obtained features integrated the information of adjacent nodes in the graph and neighboring pixels in the spatial domain. In order to reduce the computational cost of GCN, Hong et al. [28] proposed a minibatch GCN (miniGCN) that can divide an entire graph into patches for training. Instead of operating on a fixed graph, Wan et al. [29] proposed a multiscale dynamic GCN (MDGCN), which dynamically updated the multiscale graphs constructed on superpixels. They further proposed a context-aware dynamic GCN (CAD-GCN) [30] and a dual interactive GCN (DIGCN) [31] to exploit contextual relations among pixels and capture multiscale spatial information, respectively. To reduce the dependence on the affinity matrix, Ma et al. [32] introduced the hypergraph structure into GCN and proposed a feature fusion hypergraph neural network ($F^2$HNN). In order to enhance the ability to select the most significant receptive field, Ding et al. [33] designed a dynamic adaptive sampling GCN (DAS-GCN), which can update the receptive field through adaptive sampling. Chen et al. [34] proposed an automatic graph learning convolutional network (Auto-GCN), which incorporated a semisupervised Siamese network into GCNs and HSI classification networks to accomplish automatic learning and dynamic updating of the graph. To understand the global and contextual information in the graph, Ding et al. [35] proposed a semisupervised network based on the graph sample and aggregate-attention (SAGE-A) method to aggregate new neighboring nodes and capture long-range contextual relations.

To leverage the advantages of the CNN and GCN, Liu et al. [36] proposed a CNN-enhanced GCN (CEGCN) method, which can generate complementary spectral–spatial features at pixel and superpixel levels by using CNN and GCN, respectively. Dong et al. [37] fused the pixel-based CNN features and the superpixel-based graph attention network (GAT) features for HSI classification. In general, CNN can learn pixel-level features on the small regular regions, and GCN can generate superpixel-level features on large irregular regions [36]. Thus, the CNN and GCN features are complementary, and the fusing of CNN and GCN has shown better results than each network alone [36], [37].

Despite their success, most of the current GCN models are shallow (e.g., two layers in CEGCN) due to the oversmoothing problem [38]. When the number of GCN layers is increased, all nodes' representations will, finally, converge to a stationary point [39]. However, due to the complex spectral–spatial structure information contained in HSI data, a shallow GCN model cannot extract the deeper discriminative features effectively. As mentioned in [40], a deep GCN (DGCN) model is positive for classification performance. There are some methods trying to make the GCN model deeper in the field of remote sensing. For example, Bai et al. [41] proposed a deep attention GCN (DAGCN) that integrated an attention mechanism to aggregate similar spectra and designed a deep GCN that utilized a dense connection module and dilated convolution module to extract deep abstract features. Chen et al. [42] proposed a feature fusion via deep residual GCN to explore the internal relationship among HSI data. Huang et al. [43] proposed an end-to-end unsupervised GCN-based framework for superpixel-driven parcel segmentation of remote sensing images, where a deepGCN with residual connection is adopted. Learning from the CNN, these methods usually build a deep GCN based on the residual and dense connection, which still cannot solve the oversmoothing problem and may incur information loss or a decrease in accuracy [39]. There are still some algorithms coping with oversmoothing and overfitting problems from the graph structure itself. Rong et al. [39] found that oversmoothing hinders model training because a deep GCN model isolates output representations from the input features and overfitting weakens the generalization ability on a small dataset and developed a DropEdge technique to delete a certain number of edges randomly.

In recent years, to expand the differences among features of different land covers further, researchers have introduced attention mechanisms in some deep learning methods. A squeeze-and-excitation (SE) network has been proposed by Hu et al. [44], which improved classification performance by introducing the channel attention mechanism. Wang et al. [45] proposed a spatial–spectral SE network to refine the feature maps by utilizing the squeeze and excitation operations. Roy et al. [46] proposed a fused SE network to get channel attention by using global max pooling and global average pooling operations. To highlight the importance of spatial information, Woo et al. [47] proposed a convolutional block attention module (CBAM), which not only emphasizes the importance of channel attention but also obtains prominent spatial features. Fu et al. [48] proposed a dual attention network, where the position attention module (PAM) is proposed to learn the spatial interdependencies of features, and the channel attention module (CAM) is designed to model channel interdependencies. Besides, Hou et al. [49] proposed a coordinate attention (CA) to obtain accurate coordinate information. It factorizes channel attention into two 1-D feature encoding processes and aggregates features along these two spatial directions, respectively.

In this article, we propose a novel two-branch DGCN (TBDGCN) by combining the advantages of superpixel-based GCN and pixel-based CNN, which can simultaneously extract superpixel- and pixel-level features of HSIs. In the GCN branch, a deep GCN module with the DropEdge technique and residual connection is designed to alleviate oversmoothing and overfitting phenomena, which results in a deeper network structure with more than ten layers. In the CNN branch, to capture spatial positional information and channel information, we design a mixed attention mechanism to extract attention-based spectral–spatial features. The features of the GCN and CNN branches are then fused for classification. The main contributions of the proposed method are given as follows.

1) A novel TBDGCN is designed to fuse the complementary pixel-level CNN features and superpixel-level GCN features. It develops a deep GCN module with more than ten layers on large-scale irregular superpixels and

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 01:46:42 UTC from IEEE Xplore. Restrictions apply.

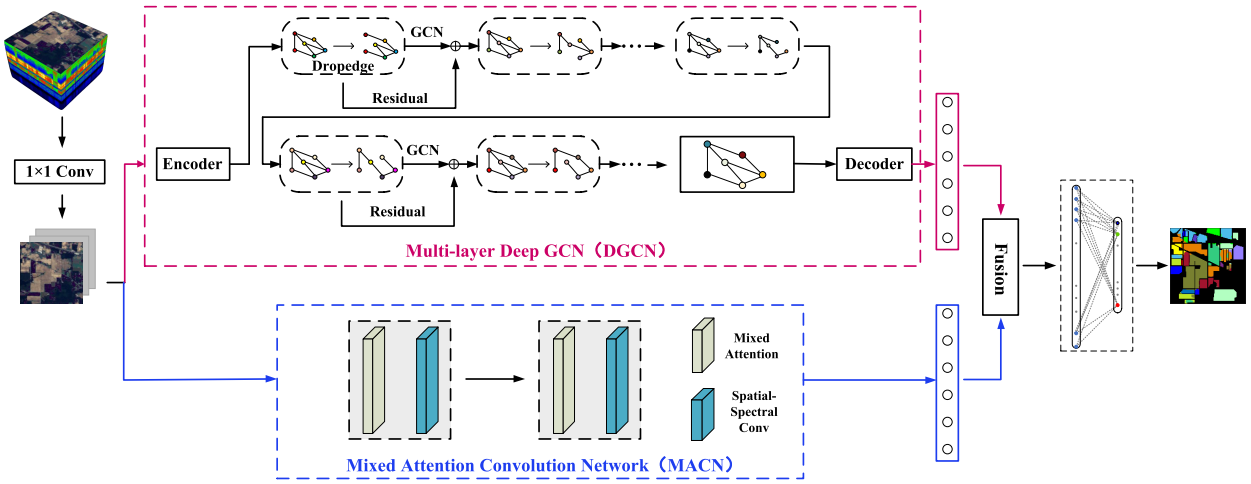



Fig. 1. Flowchart of TBDGCN. It contains two branches, i.e., multilayer DGCN and MACN. In the DGCN, the DropEdge and residual connection are used to construct a deeper graph network. In the MACN, a mixed attention mechanism is imposed on a spatial–spectral convolution, where the mixed attention mechanism consists of coordinate and SE attentions.

designs a mixed attention-based CNN module to mine intrinsic spatial–spectral features of HSI. The proposed TBDGCN can effectively capture local pixel relations in a small spatial window and long-range contextual relations among different regions.

2) A DGCN module with DropEdge and residual connection techniques is designed to alleviate the problem of overfitting and oversmoothing of deep graph networks. By deleting certain edges randomly and using residual connections, the multilayer DGCN can effectively alleviate the problem of small-sample overfitting and deep-layer oversmoothing. Experimental results verify that deepening network layers and dropping certain edges randomly are effective for HSI classification. In addition, the computational time of the proposed method is also competitive.
3) A mixed attention mechanism consisting of coordinate and SE attentions is designed to capture spatial coordinate positional information and channel information. In detail, it first extracts the spatial coordinate positional information by using two complementary 1-D global pooling operations and then captures important channel information by using SE attention. Combining the advantages of spatial coordinate and spectral SE attention, the mixed attention can mine fined spatial–spectral features of HSI.

## II. Proposed Method

The flowchart of the proposed TBDGCN is shown in Fig. 1. The original HSI is first preprocessed by $1 \times 1$ spectral convolution. Then, the preprocessed data are fed into the two branches, i.e., multilayer DGCN and mixed attention convolutional network (MACN), to extract superpixel-level features and pixel-level features, respectively. Finally, these features are fused and fed into the softmax layer to obtain the predicted label of samples.

### A. Data Preprocessing

Let an original HSI be denoted as $\mathbf{X} \in R^{H\times W\times C}$ with $H$, $W$, and $C$ corresponding to the height and width of spatial dimension and the band number of spectral dimension, respectively. Inspired by the Network-in-Network [50], we reduce noise and redundant information of the original HSI by spectral transformation. Specifically, the original data $\mathbf{X}$ are successively fed into a batch normalization (BN) layer, two 2-D CNN convolutional layers with $1 \times 1$ kernel, and a LeakyReLU activation layer. Finally, it obtains the processed data $\widehat{\mathbf{X}} \in R^{H\times W\times C_1}$ ($C_1 = 128$ in the experiments). The above processes can promote robust and discriminative spectral feature learning [36], [50].

### B. Deep Graph Convolution Network

Most of the previous GCNs only have two or three layers because the network tends to be overfitting and oversmoothing as the increase in network layers [39]. Overfitting degrades the network performance when the number of training samples is limited, and oversmoothing (i.e., all the nodes' representations will converge to an unchanging point) impedes the training of deep networks. To alleviate these problems, we construct a multilayer DGCN subnetwork, where a DropEdge technique is used to randomly remove several edges from the input graph at each training epoch [39], and a residual connection is used to avoid the vanishing gradient problem. In the DGCN, an encoder is first used to represent pixels as superpixel-based graph nodes for GCN processing, and then, the DGCN main network extracts superpixel node features, which are, finally, reconverted to pixel-level features by the decoder. As the extracted features of GCN and CNN are not incompatible, an encoder and a decoder are used to perform feature transformation between pixel- and superpixel-level features.

*1) Encoder:* HSI contains a large number of pixels, so it is unlikely to build a graph on pixel-based nodes due to the huge computational cost. To reduce the computational cost, the original HSI is first segmented into several superpixels, and the GCN is performed on the superpixel-based nodes.

The specific flowchart of the encoder module is shown in Fig. 2. First, to generate superpixels, the linear discriminant analysis (LDA) is used to reduce the dimensionality of the original HSI data, and then, the simple linear iterative

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 01:46:42 UTC from IEEE Xplore. Restrictions apply.

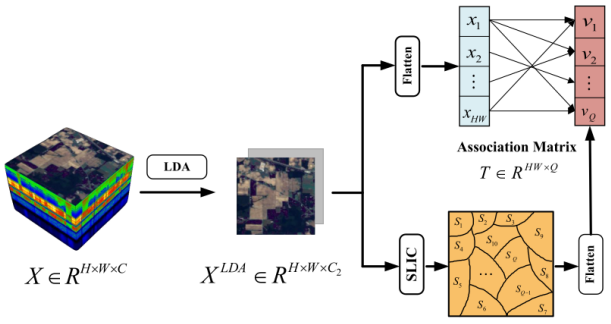



Fig. 2. Flowchart of the encoder. First, the LDA is used to reduce the dimensionality of the origin HSI data. Then, the SLIC is used to generate superpixels. Finally, an association matrix $\mathbf{T}$ between pixels and superpixels is constructed.

clustering (SLIC) [51] method is performed on the low-dimensional data $\mathbf{X}^{\mathrm{LDA}} \in R^{H \times W \times C_2}$. Finally, the HSI is partitioned into $Q$ superpixels, i.e., $\mathcal{S}_1, \ldots, \mathcal{S}_Q$, where $Q = \lfloor H \times W/100 \rfloor$.

The HSI contains $H \times W$ pixels, and each pixel belongs to one of the $Q$ superpixels. To find their relationship, an association matrix $\mathbf{T} \in R^{(HW) \times Q}$ between pixels and superpixels is constructed as

$$\mathbf{T}_{i,j} = \begin{cases} 1, & \text{if } \widetilde{\mathbf{X}}_i^{\mathrm{LDA}} \in \mathcal{S}_j \\ 0, & \text{if } \widetilde{\mathbf{X}}_i^{\mathrm{LDA}} \notin \mathcal{S}_j \end{cases} \tag{1}$$

where $\widetilde{\mathbf{X}} = \mathrm{Flatten}(\mathbf{X})$ denotes the process of flattening the 3-D data $\mathbf{X}$ by the spatial dimension and $\widetilde{\mathbf{X}}_i^{\mathrm{LDA}}$ is the $i$th pixel in $\widetilde{\mathbf{X}}^{\mathrm{LDA}} \in R^{(HW) \times C_2}$.

Based on the association matrix $\mathbf{T}$, we can encode the feature $\mathbf{X}^{\mathrm{LDA}}$ as

$$\mathbf{V} = \mathrm{Encoder}\left(\mathbf{X}^{\mathrm{LDA}}; \mathbf{T}\right) = \widehat{\mathbf{T}}^{\mathrm{T}} \widetilde{\mathbf{X}}^{\mathrm{LDA}} \tag{2}$$

where $\widehat{\mathbf{T}}$ is the normalized $\mathbf{T}$ by column and $\mathbf{V} \in R^{Q \times C_2}$ denotes the node features generated by superpixels.

Denote $\mathbf{V} = [\mathbf{v}_1, \ldots, \mathbf{v}_Q]$ and $\mathbf{v}_i$ as the feature vector of the $i$th node. Based on these superpixel node features, an undirected graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ can be constructed, where $\mathcal{V}$ and $\mathcal{E}$ are the set of nodes and edges. The edge $\mathcal{E}$ is encoded by an affinity matrix $\mathbf{A} \in R^{Q \times Q}$, which is calculated as

$$\mathbf{A}_{i,j} = \exp\left(-\frac{\|\mathbf{v}_i - \mathbf{v}_j\|^2}{\sigma^2}\right) \tag{3}$$

where $\sigma$ is a parameter and is set to 10 in the experiments.

*2) Deep Graph Convolution Network:* A graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ is defined by the nodes $\mathcal{V}$ and the edges $\mathcal{E}$, which are encoded into a node matrix $\mathbf{H}$ and affinity matrix $\mathbf{A}$ in practice. The graph Laplacian matrix $\mathbf{L}$ can be calculated by $\mathbf{L} = \mathbf{I} - \mathbf{D}^{-1/2}\mathbf{A}\mathbf{D}^{-1/2}$, where $\mathbf{I}$ is the identity matrix and $\mathbf{D}$ is a diagonal degree matrix of $\mathbf{A}$ with $\mathbf{D}_{i,i} = \sum_j \mathbf{A}_{i,j}$. The convolution on a graph can be represented as [36]

$$\mathbf{y} = \mathbf{U} g_\theta \mathbf{U}^{\mathrm{T}} \mathbf{x} \tag{4}$$

where $\mathbf{y}$ and $\mathbf{x}$ are the output and input vectors, $\mathbf{U}$ is the orthogonal matrix composed of feature vectors of $\mathbf{L}$, and $g_\theta$ is a function of the eigenvalues of $\mathbf{L}$ with respect to parameter $\theta$.

The computation of (4) is complex. In practice, the $K$th-order truncated expansion of Chebyshev polynomials is used to approximate $g_\theta$ [52]. Then, (4) can be changed to

$$\mathbf{y} = \sum_{k=0}^{K} \theta_k' T_k\left(\widehat{\mathbf{L}}\right)\mathbf{x} \tag{5}$$

where $T_k(\cdot)$ and $\theta_k'$ are the Chebyshev polynomials and Chebyshev coefficients, respectively, and $\widehat{\mathbf{L}} = (2/\lambda_{\max})\mathbf{L} - \mathbf{I}$ is the normalized $\mathbf{L}$.

Let $K = 1$ and $\lambda_{\max} = 2$; (5) can be simplified as [25]

$$\mathbf{y} = \theta\left(\mathbf{I} + \mathbf{D}^{-1/2}\mathbf{A}\mathbf{D}^{-1/2}\right)\mathbf{x}. \tag{6}$$

As a result, the convolutional operator on a graph is

$$\begin{aligned} \mathbf{H}^{\ell+1} &= \mathcal{F}^{\ell+1}\left(\mathbf{H}^\ell, \mathbf{W}^\ell\right) \\ &= h\left(\widetilde{\mathbf{D}}^{-1/2}\widetilde{\mathbf{A}}\widetilde{\mathbf{D}}^{-1/2}\mathbf{H}^\ell\mathbf{W}^\ell + b^\ell\right) \end{aligned} \tag{7}$$

where $\widetilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$, $\widetilde{\mathbf{D}}_{i,i} = \sum_j \widetilde{\mathbf{A}}_{i,j}$, $\mathbf{H}^\ell$ denotes the output feature map of the $\ell$th layer, $\mathbf{W}^\ell$ and $b^\ell$ are weights and bias to be learned, respectively, $h(\cdot)$ is the activation function, and $\mathcal{F}$ denotes the graph convolution operation.

The GCN has a strong ability to aggregate the information of neighborhood nodes. However, with the increase in the number of layers, the GCN model will be oversmoothing. That is, the representations of all nodes will converge to a stationary point, and the node discrimination will be lost [53]. To cope with this problem, inspired by Rong et al. [39] and Bai et al. [41], we utilize residual connection and DropEdge techniques to alleviate the oversmoothing and overfitting problems while increasing the depth of GCN layers.

*a) Residual connection:* The residual connection can be expressed as

$$\begin{aligned} \mathbf{H}^{\ell+1} &= \mathcal{F}^{\ell+1}\left(\mathbf{H}^\ell, \mathbf{W}^\ell\right) \\ &= \mathcal{F}^{\ell}\left(\mathbf{H}^\ell, \mathbf{W}^\ell\right) + \mathbf{H}^\ell. \end{aligned} \tag{8}$$

*b) DropEdge:* The DropEdge method is to delete a certain rate of edges of the input graph randomly at each training epoch [39]. By randomly dropping out some edges, it can increase the diversity of the input data and, meanwhile, the sparseness of node connections. The data diversity and node connection sparseness help to alleviate the overfitting and oversmoothing to some extent when the depth of GCN increases.

Let the total number of edges be $E$ and the dropping rate be $p$. The DropEdge method makes $(E \times p)$ nonzero elements of the adjacent matrix $\mathbf{A}$ be zeros. The final affinity matrix $\mathbf{A}_{\mathrm{drop}}$ is

$$\mathbf{A}_{\mathrm{drop}} = \mathbf{A} - \mathbf{A}' \tag{9}$$

where $\mathbf{A}'$ is a sparse matrix generated by a random subset of size $(E \times p)$ from original edges $\mathcal{E}$.

Based on the DropEdge, (7) can be rewritten as

$$\mathbf{H}^{(\ell+1)} = h\left(\widetilde{\mathbf{D}}^{-1/2}\widetilde{\mathbf{A}}_{\mathrm{drop}}\widetilde{\mathbf{D}}^{-1/2}\mathbf{H}^\ell\mathbf{W}^\ell + b^\ell\right). \tag{10}$$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 01:46:42 UTC from IEEE Xplore. Restrictions apply.

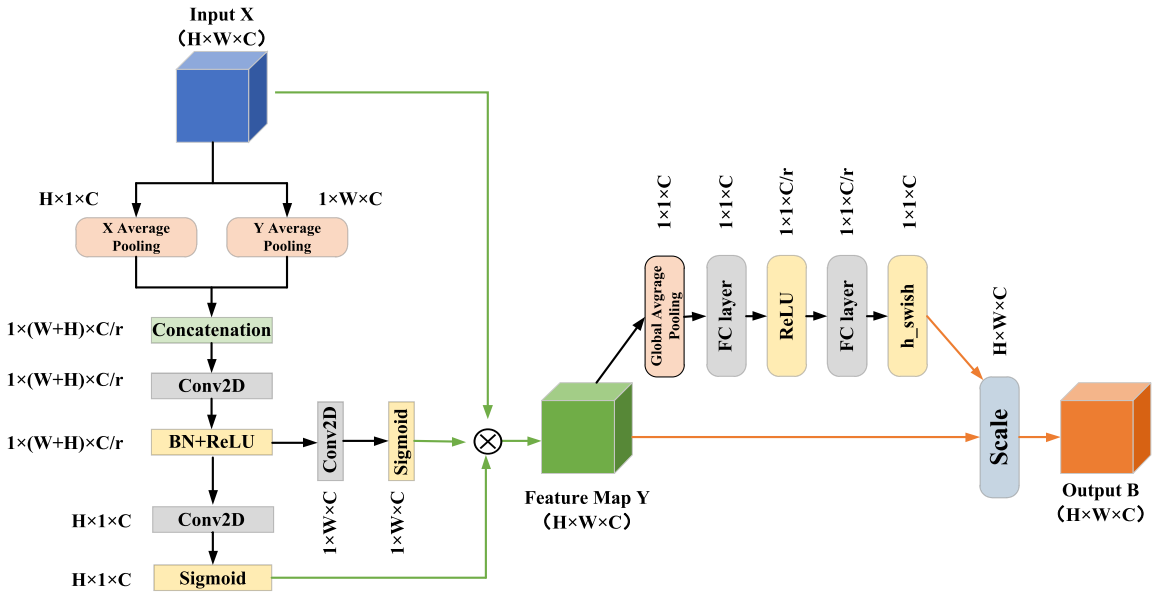



Fig. 3. Flowchart of the mixed attention mechanism.

*3) Decoder:* The task of the HSI classification is to classify pixels correctly instead of classifying superpixels based on nodes. Therefore, the features of nodes should be converted back into pixels before classification. Based on the association matrix $\mathbf{T}$, the graph decoder is

$$\mathbf{X}^* = \text{Decoder}(\mathbf{H}_{\text{out}}; \mathbf{T}) = \text{Reshape}(\mathbf{T}\mathbf{H}_{\text{out}}) \tag{11}$$

where $\mathbf{H}_{\text{out}}$ is the output feature of the DGCN and Reshape$(\cdot)$ denotes restoring the spatial dimension of the flatten data.

### C. Mixed Attention Convolutional Network

*1) Mixed Attention Mechanism:* In order to better exploit the correlation between pixels and channels, a mixed attention mechanism consisting of CA and SE attention is designed to extract spatial and spectral features of HSIs. The mixed attention mechanism is shown in Fig. 3. The CA is used to capture the detailed spatial coordinate positional information, and SE is further employed to mine channel information.

*a) Coordinate attention:* Considering spatial coordinate positional information, the CA factorizes channel attention into two parallel 1-D feature encoding processes along the two spatial directions [49].

Let $\mathbf{X} \in R^{H\times W\times C}$ be the input data and $x_c(h, w)$ be the value of the $c$th channel at location $(h, w)$. Through two pooling operations, i.e., $X$ average pooling and $Y$ average pooling, we can get the output of the $c$th channel at height $h$ as

$$z_c^h(h) = \frac{1}{W}\sum_{0\le i<W} x_c(h, i) \tag{12}$$

and the output of $c$th channel at width $w$ as

$$z_c^w(w) = \frac{1}{H}\sum_{0\le j<H} x_c(j, w). \tag{13}$$

Then, an intermediate feature map $\mathbf{f}$ can be obtained by combining the output of the $c$th channel at height and width

$$\mathbf{f} = \text{ReLU}\left(\text{BN}\left(\text{Conv}\left[\mathbf{z}_c^h, \mathbf{z}_c^w\right]\right)\right) \tag{14}$$

where [, ] is the concatenation, Conv means $1 \times 1$ convolution operation, BN denotes the batch normalization, and ReLU is the ReLU activation function. In this process, the block size can be reduced using a ratio $r$.

Next, $\mathbf{f}$ is split into two tensors $\mathbf{f}^h$ and $\mathbf{f}^w$ along the two spatial axes. Another two $1 \times 1$ convolutional layers $\text{Conv}_h$ and $\text{Conv}_w$ are used to transform them as

$$\mathbf{g}^h = \sigma\left(\text{Conv}_h\left(\mathbf{f}^h\right)\right) \tag{15}$$

$$\mathbf{g}^w = \sigma\left(\text{Conv}_w\left(\mathbf{f}^w\right)\right) \tag{16}$$

where $\sigma$ denotes the sigmoid activation function, and the outputs $\mathbf{g}^h$ and $\mathbf{g}^w$ are used as the attention weights of $H$ and $W$ spatial directions, respectively.

Finally, the output of the CA module can be calculated as

$$y_c(i, j) = x_c(i, j) \times g_c^h(i) \times g_c^w(j) \tag{17}$$

where $x_c(i, j)$ and $y_c(i, j)$ are the values of input data $\mathbf{X}$ and feature map $\mathbf{Y}$ at position $(i, j)$ and channel $c$.

It can be seen that from (12) and (13) that the CA can capture the long-range dependencies along one spatial direction and, meanwhile, preserve positional information along the other spatial direction [49].

*b) Squeeze-and-excitation:* To further exploit the global channel information, we feed the output $\mathbf{Y}$ into the next SE attention module. After the global average pooling operation, the feature map $\mathbf{Y}$ is squeezed into a vector

$$o_c = \frac{1}{H \times W}\sum_{i=1}^{H}\sum_{j=1}^{W} y_c(i, j) \tag{18}$$

where $o_c$ is the output of the $c$th channel. Then, after the fully connected layer, the activation function layer, the fully

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 01:46:42 UTC from IEEE Xplore. Restrictions apply.

connected layer, and the activation function layer, it generates output $\widetilde{\mathbf{O}} = \mathrm{FC}_2(\mathrm{ReLU}(\mathrm{FC}_1(\mathbf{O})))$, where $\mathrm{FC}_1$ and $\mathrm{FC}_2$ are two linear transformations to capture the importance of each channel.

The final output of SE is

$$\mathbf{B} = \mathbf{Y} \cdot f\left(\widetilde{\mathbf{O}}\right) \tag{19}$$

where $\cdot$ refers to channelwise multiplication and $f$ means the $h$-swish activation function.

Through this mixed attention layer, the feature expressive capabilities of CNN will be greatly improved.

*2) Spectral–Spatial Convolution:* 2-D-CNN can extract rich spatial information from HSI. However, the number of convolutional kernels may be too large due to a large number of spectral dimensions. Although 3-D CNN can reduce the number of parameters substantially, it only acts on parts of the channels in the input feature map whose receptive fields are restricted. Here, we adopt a spectral–spatial convolution [54], where each 3-D kernel is divided into a 1-D kernel and a 2-D kernel. In particular, 1-D kernels are used to realize a nonlinear transform for each pixel in the input feature map to extract spectral features and 2-D kernels serve to extract spatial features.

Let $F^k$ be the output feature map of the $k$th spectral–spatial convolutional layer, and we have

$$\mathrm{mid}_j^k(p_i) = \sigma\left(W_j^k \cdot \widetilde{F}^{k-1}(p_i) + b1_j^k\right) \tag{20}$$

$$F_j^k(p_i) = \sigma\left(\sum_{p_n \in R} K_j^k \mathrm{mid}_j^k(p_i) + b2_j^k\right) \tag{21}$$

where $\widetilde{F}^{k-1}(p_i) = \mathrm{BN}(F^{k-1}(p_i))$ is the normalized input of the $(k-1)$th layer, $\mathrm{mid}_j^k(p_i)$ denotes the output of the $j$th spectral feature map at the $k$th layer at location $p_i$, $W_j^k$ and $b1_j^k$ are the $j$th weight and bias of the spectral 1-D convolution at the $k$th layer, respectively, $K_j^k$ and $b2_j^k$ are the $j$th weight and bias of the spatial 2-D convolution at the $k$th layer, respectively, $R$ is defined as a sampling grid, and $p_n$ enumerates the positions in $R$.

### D. Feature Fusion and Classification

The features of the DGCN branch and the MACN branch can be fused by a direct concatenation operation or a linear combination as

$$\mathbf{F} = \alpha\mathbf{C} + (1-\alpha)\mathbf{G} \tag{22}$$

where $\mathbf{F}$, $\mathbf{C}$, and $\mathbf{G}$ denote the fused feature map, the MACN feature map, and the DGCN feature map, respectively, and $\alpha$ is a weight.

Finally, the softmax classifier with the cross-entropy loss function is used to classify each sample.

## III. Experiment

### A. Datasets

*1) Indian Pines (IP):* The IP dataset was collected by the Airborne Visible/Infrared Imaging Spectrometer (AVIRIS) sensor in 1992. The image size is 145 × 145 pixels with 20-m spatial resolution. The wavelength range of this data is 400–2500 nm, which contains 220 spectral bands, where 20 water absorption and noisy bands are removed. The IP data contain 16 land cover classes and 10 249 labeled samples. The pseudocolor composite image and the ground-truth map are shown in Fig. 4.

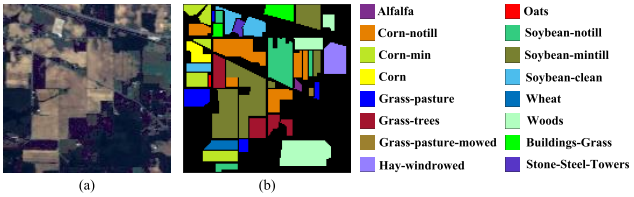



Fig. 4. (a) Pseudocolor composite image and (b) ground-truth map of IP.

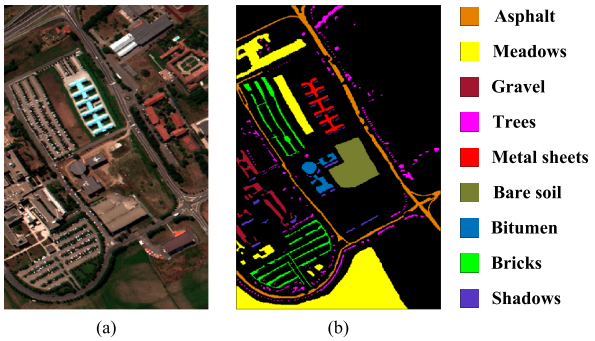



Fig. 5. (a) Pseudocolor composite image and (b) ground-truth map of UP.

*2) University of Pavia (UP):* The UP dataset was captured by the Reflective Optics System Imaging Spectrometer (ROSIS) sensor on the UP, Italy, in 2001. The image size is 610 × 340 pixels with 1.3-m spatial resolution. There are 115 spectral bands ranging from 430 to 860 nm. Among them, 12 noisy bands are removed, and the remaining 103 bands are used for classification. The UP data contain nine land cover classes and 42 776 labeled samples. The pseudocolor composite image and the ground-truth map are shown in Fig. 5.

*3) Salinas (SA):* The SA dataset was acquired by the AVIRIS on SA Valley in the United States. The image size is 512 × 217 pixels with 3.7-m spatial resolution. It has 224 spectral bands in the wavelength range of 360–2500 nm. After removing 20 noisy bands, there are 204 bands remained for research. The SA data contain 16 land cover classes and 54 129 labeled samples. The pseudocolor composite image and the ground-truth map are shown in Fig. 6.

In the experiment, only 1% labeled samples per class of the IP and 0.1% labeled samples per class of the UP and SA are selected for training and validation, respectively. Table I shows the number of training, validation, and testing samples of each class for three datasets.

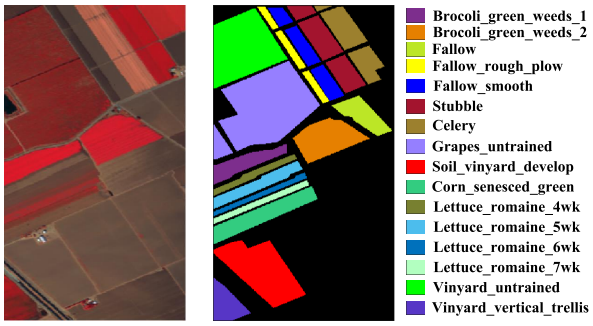
### B. Experimental Settings and Evaluation Indexes

Seven HSI classification methods are selected for comparison. They include traditional spectral-based SVM [6], CNN-based methods (i.e., 2-D-CNN, HybridSN [21], and A2S2KResNet (A2S2K for short) [55]), and GCN-based

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 01:46:42 UTC from IEEE Xplore. Restrictions apply.

TABLE I
NUMBER OF TRAINING, VALIDATION, AND TEST SAMPLES FOR EACH DATASET

| Indian Pines | | | | | University of Pavia | | | | | Salinas | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| No. | Class | Train | Val | Test | No. | Class | Train | Val | Test | No. | Class | Train | Val | Test |
| 1 | Alfalfa | 1 | 1 | 44 | 1 | Asphalt | 7 | 7 | 6617 | 1 | Brocoli_green_weeds_1 | 3 | 3 | 2003 |
| 2 | Corn-notill | 15 | 15 | 1398 | 2 | Meadows | 19 | 19 | 18611 | 2 | Brocoli_green_weeds_2 | 4 | 4 | 3718 |
| 3 | Corn-mintill | 9 | 9 | 812 | 3 | Gravel | 3 | 3 | 2093 | 3 | Fallow | 2 | 2 | 1972 |
| 4 | Corn | 3 | 3 | 231 | 4 | Trees | 4 | 4 | 3056 | 4 | Fallow_rough_plow | 2 | 2 | 1390 |
| 5 | Grass-pasture | 5 | 5 | 473 | 5 | Metal sheets | 2 | 2 | 1341 | 5 | Fallow_smooth | 3 | 3 | 2672 |
| 6 | Grass-trees | 8 | 8 | 714 | 6 | Bare soil | 6 | 6 | 5017 | 6 | Stubble | 4 | 4 | 3951 |
| 7 | Grass-pasture-mowed | 1 | 1 | 26 | 7 | Bitumen | 2 | 2 | 1326 | 7 | Celery | 4 | 4 | 3571 |
| 8 | Hay-windrowed | 5 | 5 | 468 | 8 | Bricks | 4 | 4 | 3674 | 8 | Grapes_untrained | 12 | 12 | 11247 |
| 9 | Oats | 1 | 1 | 18 | 9 | Shadows | 1 | 1 | 945 | 9 | Soil_vinyard_develop | 7 | 7 | 6189 |
| 10 | Soybean-notill | 10 | 10 | 952 | | | | | | 10 | Corn_senesced_green | 4 | 4 | 3270 |
| 11 | Soybean-mintill | 25 | 25 | 2405 | | | | | | 11 | Lettuce_romaine_4wk | 2 | 2 | 1064 |
| 12 | Soybean-clean | 6 | 6 | 581 | | | | | | 12 | Lettuce_romaine_5wk | 2 | 2 | 1923 |
| 13 | Wheat | 3 | 3 | 199 | | | | | | 13 | Lettuce_romaine_6wk | 1 | 1 | 914 |
| 14 | Woods | 13 | 13 | 1239 | | | | | | 14 | Lettuce_romaine_7wk | 2 | 2 | 1066 |
| 15 | Buildings-Grass | 4 | 4 | 378 | | | | | | 15 | Vinyard_untrained | 8 | 8 | 7252 |
| 16 | Stone-Steel-Towers | 1 | 1 | 91 | | | | | | 16 | Vinyard_vertical_trellis | 2 | 2 | 1803 |
| Total | | 110 | 110 | 10029 | | | 48 | 48 | 42680 | | | 62 | 62 | 54005 |




(a) (b)

Fig. 6. (a) Pseudocolor composite image and (b) ground-truth map of SA.

methods (i.e., miniGCN [28], FuNet [28], and CEGCN [36]). For a fair comparison, all algorithms use the parameters of the original papers in the same environment. The learning rate is set to 0.001, and the number of epochs is 300.

Three common classification indicators, i.e., overall accuracy (OA), average accuracy (AA), and $\kappa$ coefficient ($\kappa$), are used to evaluate the classification performance. The OA is calculated as the percentage of all correctly classified samples in all test samples. The AA refers to the average of the classification accuracy of each class. The $\kappa$ coefficient is an index to measure the consistency of classification results. All experimental results are the average of ten runs with randomly selected training samples.

All experiments are performed on a computer with a 2.7-GHz Intel Xeon Gold 6258R CPU and two NVIDIA GeForce RTX 2080 Ti GPUs based on Python 3.7 and Pytorch 1.7.1.

### C. Classification Results

The classification accuracy and running time of different algorithms on different datasets are shown in Tables II–IV. The classification maps on three datasets are shown in Figs. 7–9.

*1) Classification Results on IP:* The classification results of different methods on the IP dataset are shown in Table II. In the IP dataset, there are many similar classes, such as three "Corn" subclasses and three "Soybean" subclasses. The spectral curve of pixels in these subclasses is very similar. Besides, the samples in different classes are extremely unbalanced, ranging from 20 samples in "Oats" to 2455 samples in "Soybean-mintill." Due to these reasons, the classification of the IP dataset is challenging especially when the number of training samples is limited. In the experiment, only 110 training samples are used, and the comparison algorithms show poor performance with OA less than 80%. Specifically, SVM and 2DCNN have very poor results with OA less than 60% because the former does not use the rich spatial information of HSI, and the latter cannot extract good discriminating features from the spectral dimension. Compared with 2DCNN, HybridSN and A2S2K dramatically improve the classification performance due to their strong ability of spatial–spectral feature extraction with the use of 3-D convolutions. However, they cannot perform well on some classes with few pixels, e.g., Classes 1, 7, and 9. For the GCN-based methods, miniGCN simply trains the original GCN in a minibatch fashion, and its performance is not satisfactory. By fusing the GCN with 2DCNN, the FuNet improves the miniGCN to a certain extent. CEGCN fuses the pixel-level features on the small-scale regular regions in the CNN branch and superpixel-level features on large-scale irregular regions in the GCN branch and shows relatively better results than miniGCN and FuNet. By fusing pixel and superpixel features extracted from the DGCN and mixed-attention-based CNN branches, our proposed TBDGCN produces the best classification results. With the use of only 1% labeled samples per class (i.e., 110 training samples) for training, the OA of TBDGCN reaches 90.14%, which is 10% higher than the CEGCN method.

From Table II, we can also find that the accuracies of comparison methods on three "Corn" subclasses (Classes 2, 3, and 4) and three "Soybean" subclasses (Classes 10, 11, and 12)

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 01:46:42 UTC from IEEE Xplore. Restrictions apply.

TABLE II
CLASSIFICATION RESULTS ON THE IP DATASET

| No. | SVM | 2DCNN | HybridSN | A2S2K | miniGCN | FuNet | CEGCN | TBDGCN |
|---|---|---|---|---|---|---|---|---|
| 1 | 18.88 | 8.89 | 31.11 | 61.45 | 6.67 | 4.44 | 6.82 | **96.26** |
| 2 | 46.05 | 50.74 | 50.11 | 70.60 | 53.64 | 52.37 | 84.76 | **90.45** |
| 3 | 45.88 | 42.14 | 46.65 | **77.21** | 52.74 | 48.96 | 41.75 | 76.44 |
| 4 | 30.05 | 24.79 | 20.51 | 88.07 | 73.08 | 39.23 | 24.68 | **96.83** |
| 5 | 71.42 | 29.50 | 50.84 | **95.21** | 48.12 | 47.49 | 87.95 | 85.07 |
| 6 | 74.53 | 82.83 | 81.99 | 82.96 | 95.29 | 93.07 | 93.14 | **97.49** |
| 7 | 25.70 | 33.33 | 40.74 | 74.04 | 40.74 | 25.93 | 7.69 | **100** |
| 8 | 87.20 | 97.67 | **100** | 99.59 | 99.15 | 99.37 | 99.38 | 99.64 |
| 9 | 18.28 | 42.11 | 68.42 | 32.03 | 10.53 | 78.95 | 27.78 | **98.25** |
| 10 | 50.16 | 49.48 | **81.50** | 79.58 | 59.98 | 49.27 | 72.48 | 78.55 |
| 11 | 52.03 | 72.84 | 74.32 | 69.79 | 48.27 | 76.13 | **95.68** | 87.06 |
| 12 | 34.82 | 22.83 | 47.36 | 80.33 | 16.52 | 22.32 | 37.01 | **87.09** |
| 13 | 76.72 | 33.66 | 98.02 | 95.56 | 97.03 | 51,49 | **99.50** | 94.35 |
| 14 | 79.21 | 90.97 | 93.13 | 94.82 | 91.21 | 92.33 | **99.84** | 98.22 |
| 15 | 48.80 | 26.96 | 30.37 | 85.34 | 17.54 | 33.25 | 43.92 | **88.15** |
| 16 | **97.50** | 19.57 | 14.13 | 78.53 | 90.22 | 83.70 | 60.44 | 95.62 |
| OA | 55.98 | 59.68 | 67.52 | 78.54 | 60.16 | 64.56 | 79.76 | **90.14** |
| AA | 53.64 | 45.42 | 58.07 | 79.07 | 56.30 | 56.15 | 61.42 | **91.84** |
| $\kappa$ | 0.487 | 0.533 | 0.630 | 0.752 | 0.552 | 0.590 | 0.764 | **0.886** |
| Train time(s) | 2.71 | 30.22 | 49.09 | 31.76 | 3.78 | 43.22 | 6.54 | 10.31 |
| Test time(s) | 0.94 | 1.29 | 6.90 | 6.62 | 1.25 | 2.54 | 1.19 | 0.46 |

TABLE III
CLASSIFICATION RESULTS ON THE UP DATASET

| No. | SVM | 2DCNN | HybridSN | A2S2K | miniGCN | FuNet | CEGCN | TBDGCN |
|---|---|---|---|---|---|---|---|---|
| 1 | 85.83 | 76.59 | 67.04 | 66.16 | 75.47 | 81.82 | 91.16 | **91.94** |
| 2 | 73.97 | 96.71 | 94.88 | 84.51 | 79.43 | 96.17 | 97.25 | **98.22** |
| 3 | 31.14 | 67.27 | 72.57 | 44.89 | 48.33 | 60.73 | **88.26** | 81.51 |
| 4 | 70.16 | 32.45 | 43.37 | **93.37** | 72.42 | 61.83 | 68.52 | 77.23 |
| 5 | 97.27 | 88.31 | 89.95 | 94.95 | 66.64 | 93.67 | **99.97** | 98.36 |
| 6 | 45.73 | 8.70 | 49.47 | 84.50 | 15.91 | 16.32 | 92.50 | **97.61** |
| 7 | 43.21 | 36.37 | 38.63 | 65.93 | 34.56 | 36.22 | 83.05 | **84.09** |
| 8 | 64.45 | 34.42 | 27.35 | 49.79 | 66.48 | 41.76 | 76.41 | **90.50** |
| 9 | 18.28 | 89.53 | 7.51 | 62.66 | **99.47** | 95.03 | 73.20 | 74.18 |
| OA | 69.86 | 69.54 | 70.79 | 75.62 | 66.85 | 73.71 | 90.90 | **93.22** |
| AA | 67.96 | 58.93 | 54.53 | 71.86 | 62.08 | 64.84 | 86.19 | **88.18** |
| $\kappa$ | 0.583 | 0.577 | 0.606 | 0.668 | 0.563 | 0.664 | 0.866 | **0.910** |
| Train time(s) | 1.12 | 8.78 | 20.84 | 10.77 | 2.55 | 15.86 | 52.17 | 66.35 |
| Test time(s) | 3.25 | 3.94 | 12.82 | 17.84 | 31.92 | 36.36 | 9.88 | 3.56 |

are relatively poor. It is very difficult to classify all the subclasses well due to their spectral similarity. However, our proposed TBDGCN can extract the pixel- and superpixel-level features and generates relatively better results on all these subclasses. Taking into account the classification performance in the cases of small-sample-size, unbalanced classes, and spectral similar classes, our proposed method is much more robust than other methods. From the running time of

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 01:46:42 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
CLASSIFICATION RESULTS ON THE SA DATASET

| No. | SVM | 2DCNN | HybridSN | A2S2K | miniGCN | FuNet | CEGCN | TBDGCN |
|---|---|---|---|---|---|---|---|---|
| 1 | 92.70 | 94.17 | 96.16 | **99.11** | 95.66 | 68.44 | 99.06 | 97.25 |
| 2 | 98.61 | 78.29 | **100** | 99.62 | 99.41 | 97.56 | **100** | **100** |
| 3 | 75.17 | 56.53 | 85.97 | **96.28** | 54.26 | 60.99 | 88.30 | 93.36 |
| 4 | 96.79 | 80.68 | 91.45 | 93.51 | 98.64 | 92.82 | **98.66** | 80.22 |
| 5 | 91.04 | 92.71 | 87.79 | 93.35 | 98.73 | 95.93 | 96.63 | **99.63** |
| 6 | **99.87** | 99.44 | 82.60 | **99.87** | 99.39 | 98.96 | 99.45 | 97.85 |
| 7 | 94.30 | 95.89 | 71.55 | 99.86 | 99.24 | 97.06 | 98.71 | **100** |
| 8 | 65.65 | 42.68 | **100** | 81.67 | 31.57 | 37.38 | 92.15 | 97.55 |
| 9 | 95.03 | 98.72 | 46.97 | 98.04 | 97.89 | 98.82 | **100** | **100** |
| 10 | 80.87 | 66.55 | 96.25 | 97.38 | 63.23 | 84.12 | 90.89 | **97.52** |
| 11 | 58.82 | 65.48 | 46.57 | 90.72 | 90.15 | 91.37 | **94.51** | 91.45 |
| 12 | 86.41 | 58.44 | 91.83 | 94.26 | 75.69 | **98.70** | 81.28 | 85.55 |
| 13 | 81.66 | 91.26 | 90.22 | **96.65** | 30.71 | 85.03 | 92.09 | 92.89 |
| 14 | 80.08 | 71.63 | 81.71 | 89.53 | 95.13 | 83.52 | 96.62 | **97.84** |
| 15 | 48.14 | 69.09 | 43.38 | 74.20 | 72.18 | 70.94 | 79.99 | **90.47** |
| 16 | 88.64 | 72.24 | 14.13 | **97.38** | 89.92 | 80.78 | 83.98 | 93.90 |
| OA | 80.50 | 73.48 | 81.49 | 90.77 | 74.81 | 77.10 | 93.45 | **95.98** |
| AA | 83.36 | 77.71 | 81.23 | 93.77 | 80.74 | 83.90 | 93.27 | **94.72** |
| $\kappa$ | 0.782 | 0.706 | 0.783 | 0.898 | 0.719 | 0.737 | 0.927 | **0.955** |
| Train time(s) | 1.63 | 11.43 | 13.74 | 15.67 | 2.98 | 24.43 | 34.04 | 37.73 |
| Test time(s) | 4.51 | 9.83 | 17.20 | 21.41 | 53.14 | 63.58 | 4.60 | 2.25 |

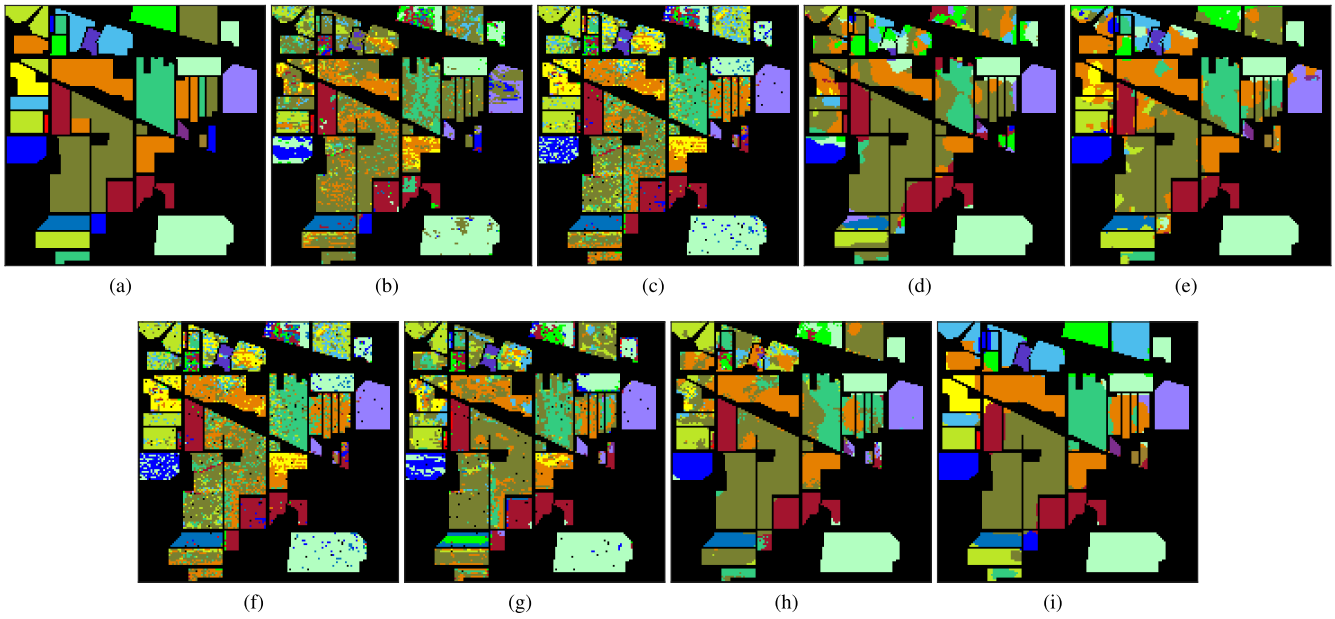



Fig. 7. Classification map on IP. (a) Ground truth. (b) SVM (55.98%). (c) 2-D-CNN (59.68%). (d) HybridSN (67.52%). (e) A2S2K (78.54%). (f) miniGCN (60.16%). (g) FuNet (64.56%). (h) CEGCN (79.76%). (i) TBDGCN (90.14%).

different methods, we can see that the computational time of our TBDGCN is competitive among GCN-based methods. Increasing the number of network layers does not dramatically increase the computational time.

From the classification maps in Fig. 7, it can be visually seen that the proposed TBDGCN not only provides smoothened classification results in large homogeneous regions, such as the Class "Woods" in the bottom right corner, and also well

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 01:46:42 UTC from IEEE Xplore. Restrictions apply.

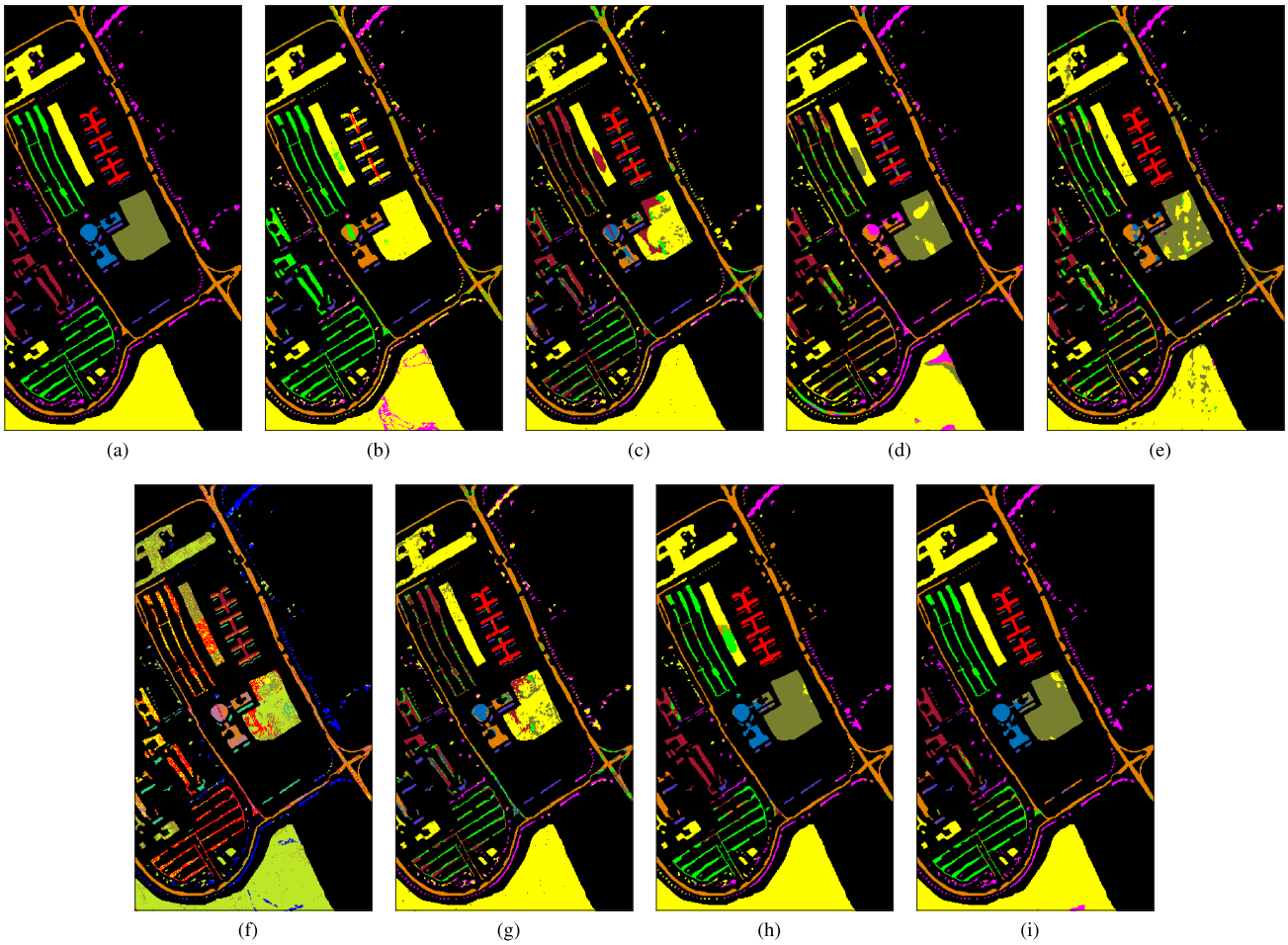



Fig. 8. Classification map on UP. (a) Ground truth. (b) SVM (69.86%). (c) 2-D-CNN (69.54%). (d) HybridSN (70.79%). (e) A2S2K (75.62%). (f) miniGCN (66.85%). (g) FuNet (73.71%). (h) CEGCN (90.90%). (i) TBDGCN (93.22%).

classifies the classes distributed in small regions, such as the Classes "Grass-pasture-mowed" and "Oats".

*2) Classification Results on UP:* Table III shows the classification results on the UP dataset. Our proposed TBDGCN method still achieves the best performance on OA, AA, and $\kappa$ coefficient, which are 93.22%, 88.18%, and 0.910, respectively. In the UP data, Class 1 "Asphalt" and Class 7 "Bitumen" are similar materials, so they are very difficult to be classified. The comparison methods provide bad results in these two classes, especially in Class 7. Our TBDGCN can extract more DGCN features and refined CNN features to mine the subtle differences between similar classes and generates the best results on these two classes. Among all the nine classes, our algorithm has the best accuracy in five of them. Taking into account both the classification accuracy and running time, our TBDGCN shows the best overall results.

Fig. 8 visually shows the classification results of each method. It can be seen that the proposed TBDGCN generates better results in the Class "Asphalt" distributed in a long and narrow region, and the Class 'Bail Soil' distributed in the center large homogeneous region.

*3) Classification Results on SA:* The classification results on the SA dataset are shown in Table IV. The number of labeled samples in the SA dataset is large, and each class has at least 900 samples. Thus, the classification of the SA is easier than the IP. The OAs of different methods are higher than 70% even with only 62 labeled samples for training. The A2S2K method shows good performance with OA of 90.77%. The miniGCN and FuNet still rely on a large number of training samples, and their performances are unsatisfactory. The CEGCN dramatically improves the miniGCN and FuNet. Our TBDGCN further improves the CEGCN. In detail, the OA, AA, and $\kappa$ values are improved by 2.43%, 1.45%, and 2.8%, respectively.

For the SA, Class 8 "Grapes_untrained" and Class 15 "Vinyard_untrained" are difficult to be classified. These two classes are adjacent classes distributed in the upper left corner of the ground-truth map, as shown in Fig. 9. From the figure, we can see that the proposed TBDGCN can well separate these two classes. In addition, for the similar Classes 11–14 (i.e., "Lettuce_romaine_4wk", "Lettuce_romaine_5wk", "Lettuce_romaine_6wk", and "Lettuce_romaine_7wk"), our TBDGCN provides much better classification results than comparison methods.

### D. Effect of the Number of Training Samples

To investigate the effect of the number of training samples, we show the classification OA of TBDGCN versus the ratio of training samples per class on the IP dataset in Fig. 10.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 01:46:42 UTC from IEEE Xplore. Restrictions apply.

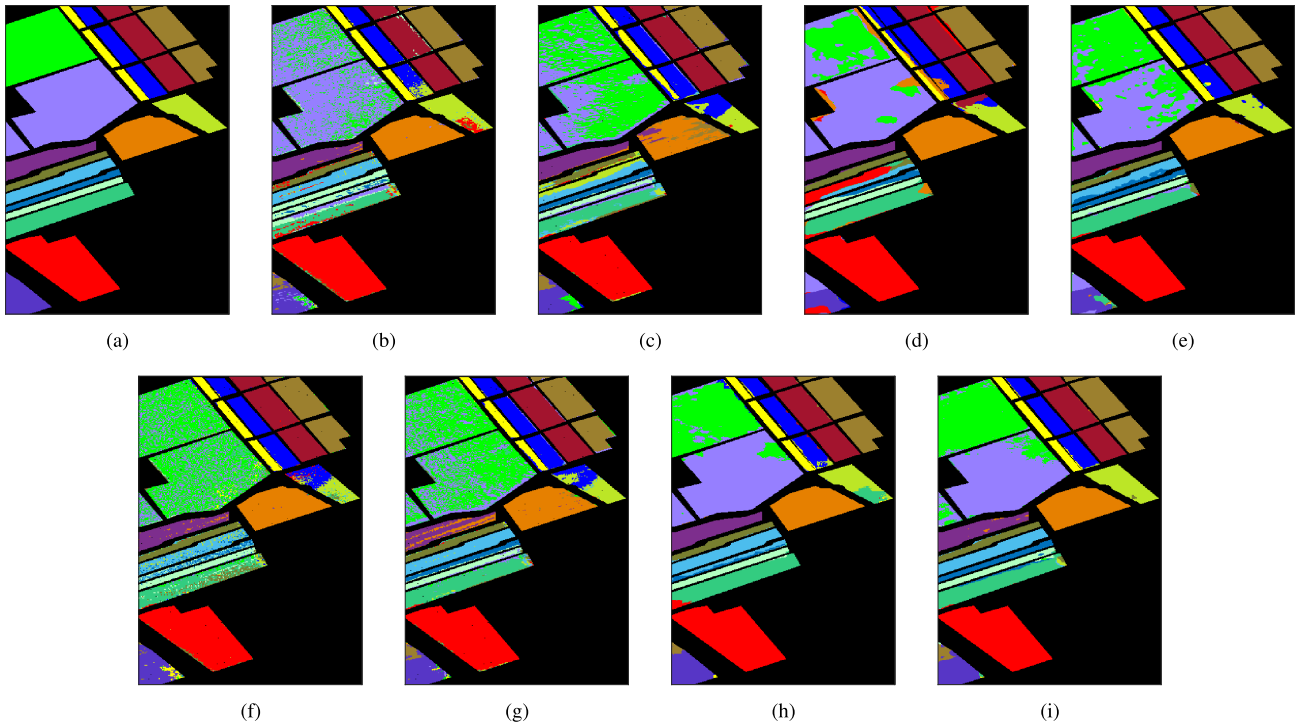



Fig. 9. Classification map on SA. (a) Ground truth. (b) SVM (80.50%). (c) 2-D-CNN (73.48%). (d) HybridSN (81.49%). (e) A2S2K (90.77%). (f) miniGCN (74.81%). (g) FuNet (77.10%). (h) CEGCN (93.45%). (i) TBDGCN (95.98%).

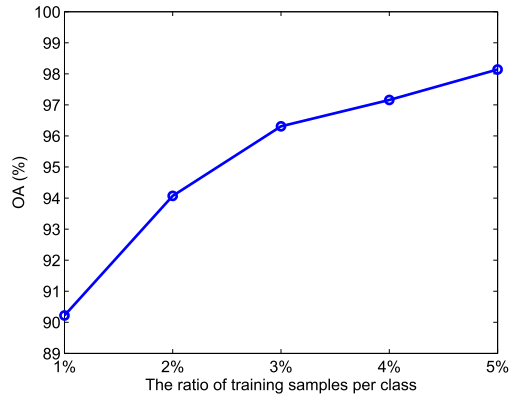



Fig. 10. OA versus the ratio of training samples per class on IP.

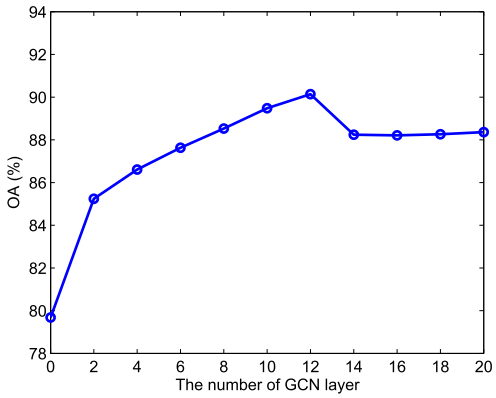



Fig. 11. OA versus the number of GCN layers on IP.

The ratio of the number of training samples ranges from 1% to 5%. It can be seen that the performance of TBDGCN gradually increases as the increase in the number of training samples.

### *E. Effect of the Number of GCN Layers*

Fig. 11 shows the OA of TBDGCN versus the number of GCN layers on the IP dataset, where the number of layer 0 means that the GCN branch is deleted (i.e., only CNN branch is used). It can be seen that the OA dramatically increases as the increase in GCN layers, reaches the maximum at 12, and then decreases slightly. It demonstrates that a moderately deep GCN structure benefits the classification, and further increasing the number of layers does not work when the OA reaches saturation. In the experiments, we set the number of GCN layers as 12 for all three datasets.

TABLE V
CLASSIFICATION RESULTS OF DIFFERENT BRANCHES ON IP

| | DGCN | MACN | TBDGCN |
|---|---|---|---|
| OA (%) | 76.15 | 79.68 | 90.14 |
| AA (%) | 73.67 | 61.69 | 91.84 |
| $\kappa$ | 0.731 | 0.764 | 0.886 |

### *F. Ablation Study*

Here, we conduct ablation experiments on the IP dataset to see the effect of different modules. The proposed TBDGCN algorithm contains two branches, i.e., the GCN branch (DGCN) and the CNN branch (MACN). It can be seen from Table V that the GCN branch or the CNN branch alone generates poor results, and the combination of them (i.e., TBDGCN) produces much better results. In the GCN branch, the SLIC algorithm is used to generate superpixels, and then, a DGCN

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 01:46:42 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
CLASSIFICATION RESULTS OF DIFFERENT SUBMODULES ON IP

| DGCN | | MACN | | OA (%) | AA (%) | $\kappa$ |
|---|---|---|---|---|---|---|
| Drop | Resi | Coor | SE | | | |
| ✓ | | ✓ | ✓ | 88.06 | 88.40 | 0.864 |
| | ✓ | ✓ | ✓ | 87.37 | 89.30 | 0.857 |
| ✓ | ✓ | | ✓ | 88.25 | 88.38 | 0.866 |
| ✓ | ✓ | ✓ | | 87.95 | 87.28 | 0.868 |
| ✓ | ✓ | | | 87.50 | 86.97 | 0.857 |
| ✓ | ✓ | ✓ | ✓ | 90.14 | 91.84 | 0.886 |

subnetwork is constructed on the superpixel nodes by using the DropEdge and residual connection techniques. Hence, the GCN branch mainly focuses on the superpixel-level features and cannot generate subtle individual features for each pixel [36]. In contrast, the CNN can learn pixel-level local spectral–spatial features but is difficult to extract large structural features due to the limited patch size (i.e., receptive field). A remote sensing image usually has complex spatial structures, and the distribution of different classes is different. Using only the pixel-level or superpixel-level features is insufficient to fully capture the spatial–spectral information of different materials. By fusing the GCN branch (DGCN) and the CNN branch (MACN), our proposed TBDGCN can take advantage of pixel- and superpixel-level features and generates much better results than the CNN or GCN alone.

Next, we further verify the effectiveness of submodules in each branch and show the corresponding results in Table VI. In the GCN branch (DGCN), the DropEdge (Drop) and residual connection (Resi) techniques are used to alleviate oversmoothing and overfitting phenomena when the number of GCN layers increases. It can be seen from the table that deleting either DropEdge or residual connection will degrade the classification performance. In the CNN branch (MACN), a mixed attention mechanism, including CA and SE attention, is used. It is clear that two attention modules are useful. If we delete both attention modules, the OA will drop by 2.6%, and AA will drop by about 5%, which indicates that the mixed attention is effective, especially for the classes with small samples.

### G. Parameters Analysis

In the proposed TBDGCN method, the DropEdge ratio $p$ in the GCN branch and the convolutional kernel size in the CNN branch need to be tuned. Here, we analyze the effect of these parameters.

*1) DropEdge Ratio:* Fig. 12 shows the effect of DropEdge ratio $p$ (i.e., the ratio of deleted edges) on the OA of TBDGCN, where $p$ changes from 0.1 to 0.9. It can be seen that the best $p$ is 0.3 for the IP and 0.4 for the UP and SA. The numbers of nodes and edges in the UP and SA are larger than the IP due to their large image sizes, so more edges in the UP and SA are deleted.

*2) Size of Convolutional Kernel:* In the CNN branch (MACN), two spatial–spectral convolutional layers are used. The size of the convolutional kernel of these two layers is chosen as 3 (i.e., 3 × 3) and 5 (i.e., 5 × 5) for considering the

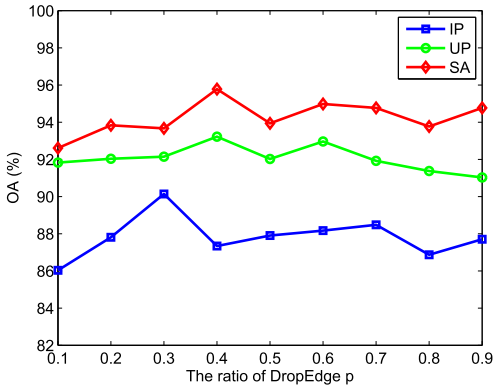



Fig. 12. OA versus the ratio of DropEdge.

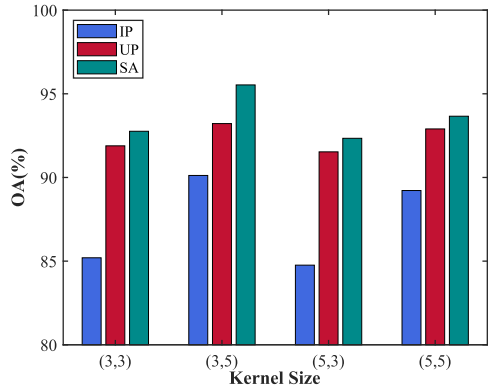



Fig. 13. OA versus the size of the convolutional kernel, where four combinations of kernel size are considered. (5, 3) means that the kernel sizes of the first and second convolutional layers in the MACN are 5 × 5 and 3 × 3, respectively.

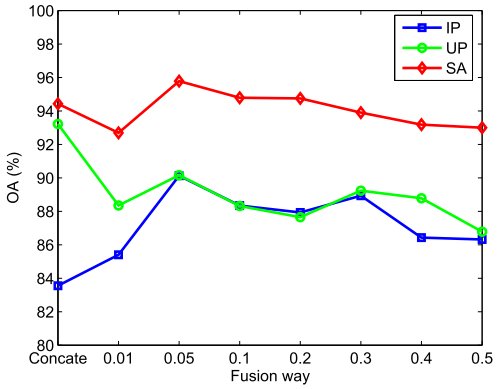



Fig. 14. Effect of different fusion ways. In the abscissa, the "Concat" means that the way of feature fusion is the concatenation, and other decimal means that the way of feature fusion is the linear combination between MACN and DGCN, and the combination coefficient $\alpha$ is decimal.

computational cost. There are four combinations, i.e., (3, 3), (3, 5), (5, 3), and (5, 5), where the numbers in the parentheses correspond to the kernel size of the first and second layers, respectively. From the results in Fig. 13, we can see that the kernel size of (3, 5) produces the best results on three datasets. In the experiments, the kernel sizes of the first and second layers are set as 3 × 3 and 5 × 5, respectively.

*3) Feature Fusion Way:* The features of GCN and CNN branches can be fused based on the concatenation or the linear combination as in (22). We investigate the effect of feature fusion way in Fig. 14, where the linear combination coefficient

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 01:46:42 UTC from IEEE Xplore. Restrictions apply.

$\alpha$ varies in the range of 0.01, 0.05, 0.1, 0.2, 0.3, 0.4, and 0.5. It can be seen that the best coefficient is $\alpha = 0.05$ in the linear weighted fusion way for all three datasets. If the concatenation fusion is also considered, the OA on the UP dataset can be further improved. In the experiments, linear weighted fusion with $\alpha = 0.05$ is used for the IP and SA, and the concatenation fusion way is used for the UP.

## IV. Conclusion

In this article, a TBDGCN model has been proposed. The TBDGCN can extract pixel- and superpixel-level features according to two parallel CNN and GCN branches (i.e., DGCN and MACN). In the GCN branch, residual connection and DropEdge strategies are employed to alleviate oversmoothing and overfitting problems and, hence, to construct a deeper graph network for discriminative feature learning. In the CNN branch, the combination of coordinate and SE attention has shown good potential in the extraction of spatial positional information and channel information. By fusing the complementary features of CNN and GCN branches, the proposed method shows excellent performance on three public HSI datasets, especially in the case of a small sample size.

## References


[1] B. Rasti et al., "Feature extraction for hyperspectral imagery: The evolution from shallow to deep: Overview and toolbox," *IEEE Geosci. Remote Sens. Mag.*, vol. 8, no. 4, pp. 60–88, Dec. 2020.

[2] J. W. P. Sun, H. Li, W. Li, X. Meng, C. Ge, and Q. Du, "Low-rank and sparse representation for hyperspectral image processing: A review," *IEEE Geosci. Remote Sens. Mag.*, vol. 10, no. 1, pp. 10–43, Jun. 2021.

[3] J. Fan, N. Zhou, J. Peng, and Y. Gao, "Hierarchical learning of tree classifiers for large-scale plant species identification," *IEEE Trans. Image Process.*, vol. 24, no. 11, pp. 4172–4184, Nov. 2015.

[4] M. Shimoni, R. Haelterman, and C. Perneel, "Hypersectral imaging for military and security applications: Combining myriad processing and sensing techniques," *IEEE Geosci. Remote Sens. Mag.*, vol. 7, no. 2, pp. 101–117, Jun. 2019.

[5] M. Fauvel, Y. Tarabalka, J. A. Benediktsson, J. Chanussot, and J. C. Tilton, "Advances in spectral–spatial classification of hyperspectral images," *Proc. IEEE*, vol. 101, no. 3, pp. 652–675, Mar. 2013.

[6] F. Melgani and L. Bruzzone, "Classification of hyperspectral remote sensing images with support vector machines," *IEEE Trans. Geosci. Remote Sens.*, vol. 42, no. 8, pp. 1778–1790, Aug. 2004.

[7] J. Li, J. Bioucas-Dias, and A. Plaza, "Spectral–spatial hyperspectral image segmentation using subspace multinomial logistic regression and Markov random fields," *IEEE Trans. Geosci. Remote Sens.*, vol. 50, no. 3, pp. 809–823, Mar. 2012.

[8] M. Belgiu and L. Drăgut, "Random forest in remote sensing: A review of applications and future directions," *ISPRS J. Photogramm. Remote Sens.*, vol. 114, pp. 24–31, Apr. 2016.

[9] G. Camps-Valls, L. Gomez-Chova, J. Muñoz-Maré, J. Vila-Francés, and J. Calpe-Maravilla, "Composite kernels for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 3, no. 1, pp. 93–97, Jan. 2006.

[10] J. Peng, Y. Zhou, and C. L. P. Chen, "Region-kernel-based support vector machines for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 53, no. 9, pp. 4810–4824, Sep. 2015.

[11] X. Cao, F. Zhou, L. Xu, D. Meng, Z. Xu, and J. Paisley, "Hyperspectral image classification with Markov random fields and a convolutional neural network," *IEEE Trans. Image Process.*, vol. 27, no. 5, pp. 2354–2367, May 2018.

[12] W. Li and Q. Du, "Gabor-filtering-based nearest regularized subspace for hyperspectral image classification," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 7, no. 4, pp. 1012–1022, Apr. 2014.

[13] S. Jia et al., "Flexible Gabor-based superpixel-level unsupervised LDA for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 12, pp. 10394–10409, Dec. 2021.

[14] M. Fauvel, J. A. Benediktsson, J. Chanussot, and J. R. Sveinsson, "Spectral and spatial classification of hyperspectral data using SVMs and morphological profiles," *IEEE Trans. Geosci. Remote Sens.*, vol. 46, no. 11, pp. 3804–3814, Nov. 2007.

[15] J. Peng, W. Sun, and Q. Du, "Self-paced joint sparse representation for the classification of hyperspectral images," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 2, pp. 1183–1194, Feb. 2019.

[16] S. Jia, S. Jiang, Z. Lin, N. Li, M. Xu, and S. Yu, "A survey: Deep learning for hyperspectral image classification with few labeled samples," *Neurocomputing*, vol. 448, pp. 179–204, Aug. 2021.

[17] Y. S. Chen, X. Zhao, and X. Jia, "Spectral–spatial classification of hyperspectral data based on deep belief network," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 8, no. 6, pp. 2381–2392, Jun. 2014.

[18] Y. Chen, H. Jiang, C. Li, X. Jia, and P. Ghamisi, "Deep feature extraction and classification of hyperspectral images based on convolutional neural networks," *IEEE Trans. Geosci. Remote Sens.*, vol. 54, no. 10, pp. 6232–6251, Oct. 2016.

[19] J. Yue, W. Zhao, S. Mao, and H. Liu, "Spectral–spatial classification of hyperspectral images using deep convolutional neural networks," *Remote Sens. Lett.*, vol. 6, no. 6, pp. 468–477, May 2015.

[20] L. Ying, H. Zhang, and S. Qiang, "Spectral–spatial classification of hyperspectral imagery with 3D convolutional neural network," *Remote Sens.*, vol. 9, no. 1, p. 67, Jan. 2017.

[21] S. K. Roy, G. Krishna, S. R. Dubey, and B. B. Chaudhuri, "HybridSN: Exploring 3-D–2-D CNN feature hierarchy for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 17, no. 2, pp. 277–281, Feb. 2020.

[22] Z. Zhong, J. Li, Z. Luo, and M. Chapman, "Spectral–spatial residual network for hyperspectral image classification: A 3-D deep learning framework," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 2, pp. 847–858, Feb. 2018.

[23] J. B. Estrach, W. Zaremba, A. Szlam, and Y. LeCun, "Spectral networks and deep locally connected networks on graphs," in *Proc. ICLR*, 2014.

[24] Y. Gu, Q. Wang, H. Wang, D. You, and Y. Zhang, "Multiple kernel learning via low-rank nonnegative matrix factorization for classification of hyperspectral imagery," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 8, no. 6, pp. 2739–2751, Jun. 2015.

[25] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," 2016, *arXiv:1609.02907*.

[26] F. Wu, A. Souza, T. Zhang, C. Fifty, T. Yu, and K. Weinberger, "Simplifying graph convolutional networks," in *Proc. ICML*, 2019, pp. 6861–6871.

[27] A. Qin, Z. Shang, J. Tian, Y. Wang, T. Zhang, and Y. Y. Tang, "Spectral–spatial graph convolutional networks for semisupervised hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 16, no. 2, pp. 241–245, Feb. 2019.

[28] D. Hong, L. Gao, J. Yao, B. Zhang, A. Plaza, and J. Chanussot, "Graph convolutional networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 7, pp. 5966–5978, Jul. 2021.

[29] S. Wan, C. Gong, P. Zhong, B. Du, L. Zhang, and J. Yang, "Multiscale dynamic graph convolutional network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 5, pp. 3162–3177, May 2019.

[30] S. Wan, C. Gong, P. Zhong, S. Pan, G. Li, and J. Yang, "Hyperspectral image classification with context-aware dynamic graph convolutional network," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 1, pp. 597–612, Jan. 2021.

[31] S. Wan, S. Pan, P. Zhong, X. Chang, J. Yang, and C. Gong, "Dual interactive graph convolutional networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5510214.

[32] Z. Ma, Z. Jiang, and H. Zhang, "Hyperspectral image classification using feature fusion hypergraph convolution neural network," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5517314.

[33] Y. Ding, J. Feng, Y. Chong, S. Pan, and X. Sun, "Adaptive sampling toward a dynamic graph convolutional network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5524117.

[34] J. Chen, L. Jiao, X. Liu, L. Li, F. Liu, and S. Yang, "Automatic graph learning convolutional networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5520716.

[35] Y. Ding, X. Zhao, Z. Zhang, W. Cai, and N. Yang, "Graph sample and aggregate-attention network for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 19, 2022, Art. no. 5504205.


Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 01:46:42 UTC from IEEE Xplore. Restrictions apply.

[36] Q. Liu, L. Xiao, J. Yang, and Z. Wei, "CNN-enhanced graph convolutional network with pixel- and superpixel-level feature fusion for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 10, pp. 8657–8671, Oct. 2021.
[37] Y. Dong, Q. Liu, B. Du, and L. Zhang, "Weighted feature fusion of convolutional neural network and graph attention network for hyperspectral image classification," *IEEE Trans. Image Process.*, vol. 31, pp. 1559–1572, 2022.
[38] M. Chen, Z. Wei, Z. Huang, B. Ding, and Y. Li, "Simple and deep graph convolutional networks," in *Proc. ICML*, 2020, pp. 1725–1735.
[39] Y. Rong, W. Huang, T. Xu, and J. Huang, "DropEdge: Towards deep graph convolutional networks on node classification," in *Proc. ICLR*, 2020, pp. 1–8.
[40] G. Li, M. Müller, A. Thabet, and B. Ghanem, "DeepGCNs: Can GCNs go as deep as CNNs?" in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 9267–9276.
[41] J. Bai, B. Ding, Z. Xiao, L. Jiao, H. Chen, and A. C. Regan, "Hyperspectral image classification based on deep attention graph convolutional network," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5504316.
[42] R. Chen, L. Guanghui, and C. Dai, "Feature fusion via deep residual graph convolutional network for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 19, pp. 1–5, 2022.
[43] F. Huang et al., "Unsupervised superpixel-driven parcel segmentation of remote sensing images using graph convolutional network," in *Proc. Companion Web Conf.*, Apr. 2022, pp. 1046–1052.
[44] J. Hu, L. Shen, and G. Sun, "Squeeze-and-excitation networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 7132–7141.
[45] L. Wang, J. Peng, and W. Sun, "Spatial–spectral squeeze-and-excitation residual network for hyperspectral image classification," *Remote Sens.*, vol. 11, no. 7, p. 884, Apr. 2019.
[46] S. K. Roy, S. R. Dubey, S. Chatterjee, and B. B. Chaudhuri, "FuSENet: Fused squeeze-and-excitation network for spectral–spatial hyperspectral image classification," *IET Image Process.*, vol. 14, no. 8, pp. 1653–1661, 2020.
[47] S. Woo, J. Park, J.-Y. Lee, and I. S. Kweon, "CBAM: Convolutional block attention module," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 3–19.
[48] J. Fu et al., "Dual attention network for scene segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 3146–3154.
[49] Q. Hou, D. Zhou, and J. Feng, "Coordinate attention for efficient mobile network design," in *Proc. CVPR*, Jun. 2021, pp. 13708–13717.
[50] M. Lin, Q. Chen, and S. Yan, "Network in network," in *Proc. ICLR*, 2014.
[51] R. Achanta, A. Shaji, K. Smith, A. Lucchi, P. Fua, and S. Süsstrunk, "SLIC superpixels compared to state-of-the-art superpixel methods," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 34, no. 11, pp. 2274–2282, Nov. 2012.
[52] D. K. Hammond, P. Vandergheynst, and R. Gribonval, "Wavelets on graphs via spectral graph theory," *Appl. Comput. Harmon. Anal.*, vol. 30, no. 2, pp. 129–150, Mar. 2011.
[53] Q. Li, Z. Han, and X.-M. Wu, "Deeper insights into graph convolutional networks for semi-supervised learning," in *Proc. AAAI*, 2018, pp. 1–8.
[54] Q. Liu, L. Xiao, F. Liu, and J.-H. Xu, "SSCDenseNet: A spectral–spatial convolutional dense network for hyperspectral image classification," *Acta Electron. Sinica*, vol. 48, no. 4, pp. 751–762, 2020.
[55] S. K. Roy, S. Manna, T. Song, and L. Bruzzone, "Attention-based adaptive spectral–spatial kernel ResNet for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 9, pp. 7831–7843, Sep. 2021.

**Linzhou Yu** received the B.S. degree in mathematics and applied mathematics from the School of Mathematics, East China University of Science and Technology, Shanghai, China, in 2020. He is currently pursuing the M.S. degree in applied statistics with the Faculty of Mathematics and Statistics, Hubei University, Wuhan, China.

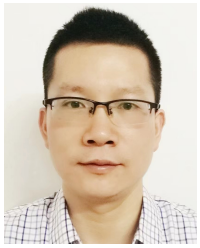
His research interests include machine learning and hyperspectral image processing.

**Jiangtao Peng** (Senior Member, IEEE) received the B.S. degree in information and computing science and the M.S. degree in applied mathematics from Hubei University, Wuhan, China, in 2005 and 2008, respectively, and the Ph.D. degree in pattern recognition and intelligent systems from the Institute of Automation, Chinese Academy of Sciences, Beijing, China, in 2011.

He is currently a Professor with the Faculty of Mathematics and Statistics, Hubei University. His research interests include machine learning and hyperspectral image processing.

**Na Chen** received the B.S. degree in information and computing science and the M.S. degree in applied mathematics from Hubei University, Wuhan, China, in 2006 and 2009, respectively, and the Ph.D. degree in computational mathematics from the Huazhong University of Science and Technology, Wuhan, in 2012.

She is currently an Associate Professor with the Faculty of Mathematics and Statistics, Hubei University. Her research interests include statistical learning theory and image processing.



**Weiwei Sun** (Senior Member, IEEE) received the B.S. degree in surveying and mapping and the Ph.D. degree in cartography and geographic information engineering from Tongji University, Shanghai, China, in 2007 and 2013, respectively.

From 2011 to 2012, he studied at the Department of Applied Mathematics, University of Maryland, College Park, MD, USA, where he worked as a Visiting Scholar with the famous professor John Benedetto to study the dimensionality reduction of hyperspectral image. From 2014 to 2016, he studied at the State Key Laboratory for Information Engineering in Surveying, Mapping and Remote Sensing (LIESMARS), Wuhan University, Wuhan, China, where he worked as a Post-Doctoral Researcher to study intelligent processing in hyperspectral imagery. From 2017 to 2018, he worked as a Visiting Scholar at the Department of Electrical and Computer Engineering, Mississippi State University, Mississippi State, MS, USA. He is currently a Full Professor with Ningbo University, Ningbo, Zhejiang, China. He has published more than 70 journal articles. His research interests include hyperspectral image processing with manifold learning, anomaly detection, and target recognition of remote sensing imagery using compressive sensing.

**Qian Du** (Fellow, IEEE) received the Ph.D. degree in electrical engineering from the University of Maryland at Baltimore County, Baltimore, MD, USA, in 2000.

She is currently a Bobby Shackouls Professor with the Department of Electrical and Computer Engineering, Mississippi State University, Mississippi State, MS, USA. Her research interests include hyperspectral remote sensing image analysis and applications, pattern classification, data compression, and neural networks.

Dr. Du is a fellow of the SPIE-International Society for Optics and Photonics. She is a member of the IEEE Periodicals Review and Advisory Committee. She was a recipient of the 2010 Best Reviewer Award from the IEEE Geoscience and Remote Sensing Society (GRSS). She has served as the Co-Chair of the Data Fusion Technical Committee of the IEEE GRSS from 2009 to 2013. She was the Chair of the Remote Sensing and Mapping Technical Committee of the International Association for Pattern Recognition from 2010 to 2014. She was the General Chair of the fourth IEEE GRSS Workshop on Hyperspectral Image and Signal Processing: Evolution in Remote Sensing held in Shanghai, China, in 2012. She has served as an Associate Editor for the IEEE JOURNAL OF SELECTED TOPICS IN APPLIED EARTH OBSERVATIONS AND REMOTE SENSING (JSTARS), the *Journal of Applied Remote Sensing*, and the IEEE SIGNAL PROCESSING LETTERS. From 2016 to 2020, she was the Editor-in-Chief of the IEEE JSTARS.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 01:46:42 UTC from IEEE Xplore. Restrictions apply.